

Une requête, trois conteneurs, deux volumes

Support d'étude pour la soutenance : l'architecture complète, l'ordre pour l'apprendre, les tests de la fiche d'évaluation et les pièges rencontrés en route.

VM HÔTE — SEUL LE PORT 443 EST PUBLIÉ

Navigateur

<https://mcastrat.42.be>

HTTPS · 443 · TLSv1.3 ↓

RÉSEAU DOCKER « INCEPTION » (BRIDGE + DNS INTERNE)

nginx

déchiffre le TLS, sert le statique, délègue les .php

FastCGI · wordpress:9000 ↓

wordpress

php-fpm exécute le code WordPress

SQL · mariadb:3306 ↓

mariadb

stocke articles, comptes, commentaires

wp_data → /home/mcastrat/data/wordpress —
monté dans **wordpress** et **nginx** (/var/www/
html)

mariadb_data → /home/mcastrat/data/mariadb
— monté dans **mariadb** (/var/lib/mysql)

01 — LA STRUCTURE

10 fichiers, chacun un seul rôle

Question officielle de la fiche : « la pertinence de la structure ». Réponse : orchestration (compose), construction (un dossier autonome par service), configuration (conf/ et tools/) sont séparées — chaque service se comprend et se rebuild indépendamment.

Makefile		l'entrée unique : <code>make</code> crée les dossiers de données puis <code>docker compose up -d --build</code>
srcs/docker-compose.yml		déclare les 3 services, le réseau, les volumes bindés et les secrets
srcs/.env		variables non sensibles : domaine, noms de base et d'utilisateurs, emails
mariadb/Dockerfile	MARIADB	debian:bookworm + mariadb-server, purge de /var/lib/mysql
mariadb/conf/50-server.cnf	MARIADB	<code>bind-address = 0.0.0.0</code> : accepter les connexions du réseau Docker
mariadb/tools/init.sh	MARIADB	au 1er démarrage : crée la base, wpuser, le mot de passe root
wordpress/Dockerfile	WORDPRESS	php-fpm 8.2 + php-mysql + wp-cli (aucun nginx : point éliminatoire)
wordpress/conf/www.conf	WORDPRESS	php-fpm écoute en TCP sur le port <code>9000</code>
wordpress/tools/setup.sh	WORDPRESS	attend la base, installe WordPress via wp-cli au 1er démarrage
nginx/Dockerfile	NGINX	nginx + certificat auto-signé généré au build avec openssl
nginx/conf/inception.conf	NGINX	server block : 443 seul, TLS 1.2/1.3, <code>fastcgi_pass wordpress:9000</code>

02 – L'ORDRE D'ÉTUDE

Apprendre dans l'ordre de démarrage

C'est l'ordre du `depends_on` : chaque service ne se comprend qu'avec celui d'en dessous.

ÉTAPE 1 · LE SOCLE

MariaDB

Le plus simple, et il introduit *le* pattern du projet (encadré ci-dessous).

`if [ ! -d /var/lib/mysql/mysql ]` = la persistance : on n'initialise que si le volume est vierge.

- Pourquoi `FLUSH PRIVILEGES;` en tête du SQL d'init ?
- Pourquoi supprimer les utilisateurs anonymes ?
- Pourquoi `bind-address = 0.0.0.0` ?

ÉTAPE 2 · LE GROS MORCEAU

WordPress / php-fpm

Même squelette que MariaDB, trois nouveautés : la boucle `until` qui attend la base (car `depends_on` attend le *démarrage* du conteneur, pas que la base soit *prête*), wp-cli (`download` / `config create` / `core install`), et `listen = 9000` — retiens ce port.

- Que fait php-fpm exactement ? (il exécute du PHP, il ne parle pas HTTP)
- Pourquoi zéro nginx dans cette image ?
- D'où viennent les mots de passe ? (Docker secrets, jamais dans l'image)

ÉTAPE 3 · LA PRISE MURALE

NGINX

Court, mais il branche tout : `listen 443 ssl` uniquement (pas de bloc port 80 → connexion refusée), certificat auto-signé au build, et `fastcgi_pass wordpress:9000;` — le DNS du réseau Docker résout `wordpress`, et 9000 est le port vu à l'étape 2.

- Qui sert les images et le CSS ? (nginx lui-même, via le volume partagé)
- Qui sert les .php ? (php-fpm, via FastCGI)
- Pourquoi le certificat peut être auto-signé ?

si les données n'existent pas → initialiser, puis exec le serveur en avant-plan

Le pattern des deux entrypoints. Le `if` donne la persistance (on ne réinstalle jamais deux fois), le `exec` fait du serveur le PID 1 — aucun processus en arrière-plan, aucune boucle infinie : exactement ce que la fiche vérifie.

Le trajet d'une requête

L'ordre de démarrage explique comment c'est construit ; l'ordre de requête explique comment ça marche. C'est presque toujours cette histoire-là qu'on te demandera de raconter.

- 1 Le navigateur**
ouvre `https://mcastrat.42.be` — le port 443 est le seul publié par la stack.
- 2 nginx**
termine la connexion TLS (v1.2/v1.3, certificat auto-signé). Fichier statique ? il le sert directement depuis le volume `wp_data`.
- 3 nginx**
pour un `.php` : transmet la requête en FastCGI à `wordpress:9000` — nom résolu par le DNS interne du réseau bridge.
- 4 php-fpm**
exécute le code WordPress (`index.php` et compagnie) depuis le même volume partagé.
- 5 MariaDB**
reçoit les requêtes SQL de WordPress sur `mariadb:3306` avec le compte `wpuser`, et lit/écrit dans `mariadb_data`.
- 6 La réponse**
HTML remonte : `php-fpm` → `nginx` qui la chiffre en TLS → navigateur.

04 — LA FICHE D'ÉVAL

Ce que l'évaluateur va tester (tout est vérifié ✓)

Chaque ligne a été rejouée sur la stack le 11/07. « Éliminatoire » = la soutenance s'arrête si ça échoue.

TEST	COMMANDE	ATTENDU
Port 80 fermé · éliminatoire	<code>curl http://mcastrat.42.be</code>	connexion refusée
HTTPS + TLS · éliminatoire	<code>openssl s_client -connect mcastrat.42.be:443</code>	TLSv1.3
Site installé · éliminatoire	ouvrir <code>https://mcastrat.42.be</code>	le site, pas l'installateur
	<code>docker exec -it mariadb mariadb -u root</code>	Access denied

root sans mot de passe · éliminatoire		
Login wpuser + base non vide	<code>mariadb -u wpuser -p wordpress</code> puis <code>SHOW TABLES;</code>	tables wp_*
Volumes dans /home/login/data	<code>docker volume inspect srcs_wp_data</code>	device conforme
Réseau visible	<code>docker network ls</code>	srcs_inception
Admin sans « admin » dans le nom	login wp-admin	mcastrat
Persistance	reboot VM puis <code>make</code>	données intactes
Dockerfiles propres · éliminatoire	lecture : pas de <code>tail -f</code> , ni sleep infini, ni processus en fond	exec en PID 1

05 – TES ANECDOTES

Trois bugs vécus – de l'or en soutenance

Réponse toute prête à « quelles difficultés as-tu rencontrées ? ». Chacun prouve une vraie compréhension de Docker.

① Le volume pré-rempli

Un volume nommé vide est rempli avec le contenu de l'image au premier montage. La base pré-installée par le paquet Debian court-circuitait le script d'init.

Fix : `rm -rf /var/lib/mysql/*` et `/var/www/html/*` au build.

② Le bootstrap sans privilèges

`mysqld --bootstrap` tourne en mode skip-grant-tables : les CREATE USER étaient ignorés sans erreur.

Fix : `FLUSH PRIVILEGES;` en tête du SQL d'init.

③ Les utilisateurs anonymes

mariadb-install-db crée `'@'localhost'`, plus spécifique que `'wpuser'@'%'` : le login local de wpuser était refusé — alors que WordPress, lui, passait.

Fix : `DROP USER '@'localhost';` dans l'init.

06 — QUESTIONS DE COURS

Auto-test : réponds avant d'ouvrir

▼ Docker vs machine virtuelle : la différence ?

Une VM émule une machine entière avec son propre noyau. Un conteneur n'est qu'un processus isolé (namespaces + cgroups) qui **partage le noyau de l'hôte**. Conséquence : démarrage en secondes et quelques Mo de surcoût, contre des minutes et des Go pour une VM — mais isolation moins forte.

▼ Image vs conteneur ? Et le rôle de docker-compose ?

Une image est une recette figée en couches, décrite par un Dockerfile. Un conteneur est une instance vivante d'une image. Compose est un orchestrateur local : le YAML déclare services, réseau, volumes et secrets, et `docker compose up -d --build` construit et démarre le tout dans l'ordre des `depends_on`.

▼ Une image avec ou sans compose, quelle différence ?

Aucune : c'est la même image. Sans compose il faudrait tout faire à la main — `docker build`, `docker network create`, un `docker run` à dix options par conteneur. Compose rend l'ensemble déclaratif et reproductible.

▼ C'est quoi, le réseau Docker de ce projet ?

Un réseau bridge privé nommé `inception`. Docker y fournit un DNS interne : nginx joint `wordpress:9000` et WordPress joint `mariadb:3306` par leur nom de service. Seul nginx publie un port vers l'extérieur (443) — la base et php-fpm sont injoignables depuis l'hôte. `network: host` et `links:` sont interdits par le sujet.

▼ Pourquoi une boucle d'attente dans setup.sh alors que depends_on existe ?

`depends_on` attend que le conteneur mariadb soit *démarré*, pas que le serveur SQL soit *prêt* à accepter des connexions. La boucle réessaie (30 fois maximum — bornée, donc pas une boucle infinie) jusqu'à ce qu'un vrai `SELECT 1` passe.

▼ Où vivent les mots de passe ?

Dans `secrets/` à la racine, ignoré par git. Compose les monte en fichiers dans `/run/secrets/` et les scripts les lisent avec `cat`. Jamais dans le Dockerfile, jamais en ARG de build (ça resterait lisible dans `docker history`), jamais commités.

▼ Pourquoi debian:bookworm ?

Le sujet impose l'avant-dernière version stable. Debian 13 « trixie » étant la stable actuelle, l'avant-dernière est la 12 « bookworm ». L'image est en plus multi-arch : le même Dockerfile build sur la VM ARM64 de la maison et sur la VM x86_64 de l'école.

▼ Comment marche la persistance ?

Les deux volumes sont bindés sur `/home/mcastrat/data/` sur l'hôte. Les scripts d'init ne font quelque chose que si le volume est vierge (`if [ ! -d ... ]` / `if [ ! -f wp-config.php ]`). Après un reboot, `restart: always` relance les conteneurs, qui retrouvent leurs données et sautent l'initialisation.

07 – AVANT LE JOUR J

À ne pas oublier

Recréer secrets/ après le clone

Le dossier est gitignoré (exigence du sujet) : sur la station d'éval, le build échouera tant que `db_password.txt`, `db_root_password.txt` et `credentials.txt` (ligne 1 : mot de passe admin, ligne 2 : mot de passe user) n'existent pas.

La commande de purge de la fiche – jamais sur la VM maison

`docker stop $(docker ps -qa) ; docker rmi -f ...` détruit *tous* les conteneurs de la machine, y compris ceux des autres projets. À réserver à la VM d'évaluation vierge.

Changer les mots de passe WordPress

Actuellement « Admin » et « Guest » — ça marche, mais c'est gênant à montrer. Modifier credentials.txt puis `make re`.

Stack revalidée de bout en bout le 11/07/2026 : build via make, 443 seul, TLSv1.3, root sans mot de passe refusé, login wpuser OK, persistance après down/up.